



FACULTAD DE INGENIERÍA

Machine Learning en Producción

Obligatorio: Predicción de ratings de películas IMDb

2026

Alumnos:

Pablo Durán - 270956

Marcos Mussio - 362839

Natalia Campiglia - 349251



[mlp-obligatorio-imdb](#)

Última actualización: 9/07/2026

Índice

1. Resumen	4
2. Objetivo del proyecto	4
3. Arquitectura general	5
4. Ingesta de datos	6
4.1. Logs y diagnóstico del scraper	7
5. Almacenamiento y versionado de datos	8
6. ETL	9
6.1. Logs y diagnóstico del ETL	11
7. Feature engineering	12
7.1. Feature engineering del entrenamiento actual	13
8. Entrenamiento del modelo	14
8.1. Publicación en W&B	15
9. Prevención de data leakage y training-serving skew	16
9.1. Data leakage	16
9.2. Training-serving skew	17
10. Trazabilidad de ML	17
11. Serving del modelo	18
11.1. Predicciones en landing page	18
11.2. Predicciones batch	19
12. Containerización	19
13. Infraestructura como código	19

14.Gestión de secretos	20
15.Observabilidad y monitoreo	20
16.Automatización	21
17.Conceptos de clase aplicados	21
18.Trabajo pendiente	22
19.Conclusiones	23

1. Resumen

El proyecto implementa una solución de Machine Learning en Producción para predecir el rating de películas de IMDb. La solución cubre el ciclo completo desde la recolección de datos hasta el despliegue de una aplicación web en AWS.

El flujo principal es:

1. Recolectar datos de películas y reviews desde IMDb.
2. Persistir datos crudos en S3 en formato Parquet.
3. Ejecutar un proceso ETL para construir datasets de entrenamiento e inferencia.
4. Entrenar, evaluar y registrar un modelo de predicción.
5. Exponer la solución mediante una API FastAPI containerizada.
6. Desplegar la aplicación en AWS ECS Fargate usando Terraform.

La solución busca aplicar conceptos de MLOps vistos en clase: automatización de pipelines, versionado de datos, separación entre datos crudos y procesados, tracking de modelos, model registry, despliegue reproducible, gestión de secretos y monitoreo básico. Estas prácticas son relevantes porque los sistemas de Machine Learning suelen acumular deuda técnica no solo en el modelo, sino también en los datos, pipelines, configuración y serving [3, 2].

2. Objetivo del proyecto

El objetivo es construir una aplicación capaz de estimar el rating esperado de una película a partir de atributos disponibles en IMDb, como año, duración, metascoring, géneros, director, elenco principal y señales derivadas de reviews de usuarios.

Además del modelo predictivo, el foco del obligatorio está en construir una arquitectura productiva alrededor del modelo. Por eso el proyecto no se limita al entrenamiento, sino que incluye

ingesta de datos, procesamiento, almacenamiento, despliegue cloud e infraestructura como código.

3. Arquitectura general

La arquitectura propuesta se compone de los siguientes bloques:

Componente	Tecnología	Responsabilidad
Scraper	Python, Playwright	Extraer películas y reviews desde IMDb.
Data lake	AWS S3	Almacenar datos crudos, procesados y de inferencia.
ETL	Python, pandas, pyarrow	Transformar datos crudos en datasets listos para entrenamiento e inferencia.
Tracking y registry	Weights & Biases	Registrar experimentos, métricas y modelos.
API	FastAPI	Exponer endpoints para estado, películas, modelo y predicción.
Container	Docker	Empaquetar la aplicación para ejecución reproducible.
Infraestructura	Terraform	Crear recursos cloud de forma declarativa.
Despliegue	AWS ECS Fargate, ECR, ALB	Ejecutar la API en producción.
Logs	AWS CloudWatch	Observar comportamiento de la aplicación.

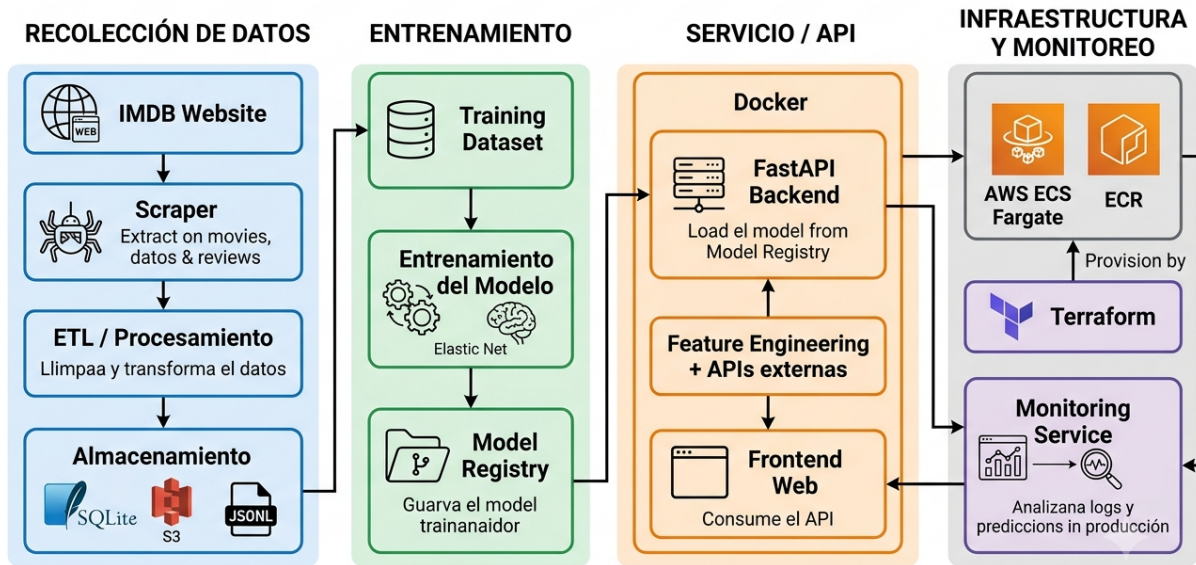


Figura 1: Arquitectura general de la solución: recolección de datos, entrenamiento, servicio/API e infraestructura cloud.

4. Ingesta de datos

La ingesta se realiza con un scraper de IMDb implementado en Python. El scraper obtiene información de películas y reviews, y guarda los resultados en formato local y en S3.

Datos recolectados para películas:

- Identificador IMDb.
- Título.
- Año.
- Rating IMDb.

- Cantidad de votos.
- Géneros.
- Director/es.
- Reparto principal.
- Duración.
- Certificado de edad.
- Metascore.
- Sinopsis.

Datos recolectados para reviews:

- Película asociada.
- Usuario.
- Rating de la review.
- Texto de la review.
- Votos de utilidad.

El scraper contempla que IMDb puede requerir login para acceder a reviews completas. Para eso existe un flujo manual que guarda una sesión local y permite reutilizarla en corridas posteriores.

4.1. Logs y diagnóstico del scraper

El scraper también incorpora logs para seguir la corrida y detectar problemas durante la extracción. Se usa un logger propio que escribe tanto en consola como en archivos diarios dentro de `data/logs/`. Al comenzar, registra el rango de IDs aleatorios que va a intentar, la cantidad máxima de películas y la cantidad máxima de reviews por película. Esto permite reproducir el contexto de

una corrida y entender si el volumen obtenido coincide con la configuración esperada.

Durante la autenticación, los logs indican si se reutilizó una sesión guardada, si fue necesario iniciar sesión nuevamente o si el login falló. Esto es importante porque, cuando IMDb bloquea el acceso a la página completa de reviews, el scraper puede quedar limitado a reviews destacadas. Registrar ese estado ayuda a explicar por qué una corrida puede traer menos reviews que otra.

Mientras recorre títulos, el scraper informa cuando descarta un ID porque no corresponde a una película, cuando no encuentra título, o cuando ocurre un error al extraer una película o sus reviews. También registra el progreso de guardado con el ID de IMDb, la cantidad de películas acumuladas y cuántas reviews se obtuvieron. Al final imprime una tabla con las películas guardadas ordenadas por cantidad de reviews y cuenta cuántas películas tienen rating pero no reviews. Esta salida sirve para análisis rápido: permite detectar películas sin señal textual, problemas de acceso a reviews o una muestra demasiado pobre para el ETL.

Finalmente, si S3 está configurado, el scraper registra la subida de los Parquet crudos con la cantidad de películas, cantidad de reviews y `run_id`. Si falla el upload, el log lo deja explícito, lo que facilita distinguir entre un problema de scraping y un problema de credenciales o conectividad con AWS.

5. Almacenamiento y versionado de datos

Los datos se almacenan en S3 en formato Parquet. Se eligió Parquet porque es un formato columnar, eficiente para lectura y escritura de datos tabulares, y se integra bien con pandas, pyarrow y servicios cloud como S3. Además, comprime mejor que formatos de texto como CSV o JSON y conserva tipos de datos de forma más robusta, algo útil para una capa raw que luego será procesada por el ETL.

Cada corrida del scraper genera archivos con timestamp, evitando sobrescribir datos anteriores.

```
s3://mlp-imdb-data/  
imdb/
```



```
movies/  
  scraped_date=YYYY-MM-DD/  
    run_YYYYMMDD_HHMMSS.parquet  
reviews/  
  scraped_date=YYYY-MM-DD/  
    run_YYYYMMDD_HHMMSS.parquet  
processed/  
  training_dataset.json  
inference/  
  inference_dataset.json
```

Esta organización permite aplicar un concepto importante de MLOps: trazabilidad de datos. Si un modelo fue entrenado con determinada versión del dataset, se puede identificar de qué corridas del scraper provienen los datos.

6. ETL

El proceso ETL transforma los datos crudos del scraper en un dataset consolidado por película. En la implementación actual, el ETL lee datos desde S3, deduplica películas, agrega una señal de sentimiento a partir de reviews y genera un JSON procesado que luego consume el entrenamiento.

Los pasos principales son:

1. Validar que existan objetos en los prefijos `movies/` y `reviews/` de S3.
2. Leer todos los Parquet crudos de películas y reviews desde S3.
3. Deduplicar películas repetidas por `imdb_id`, conservando la versión con `scraped_date` más reciente.
4. Calcular `review_agg`, una tabla agregada de sentimiento promedio por película.
5. Unir películas con `review_agg` mediante `imdb_id`.

6. Eliminar filas sin `imdb_rating`, porque no pueden usarse para entrenamiento supervisado.
7. Generar registros limpios con tipos JSON válidos, rating normalizado a escala 0–1 y listas normalizadas.
8. Guardar `training_dataset.json` localmente y subirlo a S3:

```
s3://mlp-imdb-data/imdb/processed/training_dataset.json
```

El punto más importante del ETL es la construcción de `review_agg`. El scraper guarda reviews a nivel de comentario, por lo que una misma película puede tener varias filas en el dataset de reviews. Para convertir esa información en una única feature por película, el ETL ejecuta la función `_sentimiento_vader_por_pelicula`.

La herramienta utilizada para el análisis de sentimiento es VADER, mediante la librería `vaderSentiment`. VADER es un analizador basado en léxico y reglas: no requiere entrenar un modelo propio, sino que asigna polaridad a partir de palabras, intensificadores, negaciones y otros patrones simples del lenguaje. Para este proyecto resulta adecuado por tres motivos. Primero, las reviews de usuarios suelen expresar opinión de forma directa, por lo que una señal de polaridad general es útil como feature agregada. Segundo, es liviano y reproducible: puede ejecutarse dentro del ETL sin infraestructura adicional ni dependencias de modelos grandes. Tercero, devuelve un score numérico fácil de integrar con el resto del dataset.

VADER entrega varios puntajes, pero en este ETL se usa `compound`, que resume la polaridad global del texto en un valor entre -1 y 1. Un valor cercano a -1 indica una review predominantemente negativa, un valor cercano a 1 indica una review positiva y valores alrededor de 0 indican neutralidad o mezcla de polaridades. Como el target `imdb_rating` se normaliza a escala 0–1, también se normaliza el sentimiento para mantener una escala comparable.

Primero se valida que existan las columnas `movie_imdb_id` y `review_text`. Si no hay reviews, o si faltan esas columnas, se devuelve un DataFrame vacío con las columnas esperadas: `imdb_id` y `reviews`. Esto permite que el resto del pipeline siga funcionando aunque no haya textos disponibles.

Luego se toma una copia de `movie_imdb_id` y `review_text`, se reemplazan valores nulos por

strings vacíos y se eliminan reviews cuyo texto queda vacío después de aplicar `strip`. Sobre cada texto válido se aplica VADER y se guarda el puntaje `compound`. Ese valor se normaliza a escala 0–1 con:

```
reviews["reviews"] = (reviews["vader_compound"] + 1) / 2
```

Finalmente, las reviews se agrupan por película y se calcula el promedio del sentimiento normalizado. Ese resultado es `review_agg`: una tabla con una fila por película y una columna `reviews`.

Luego, `review_agg` se une al dataset de películas con un `left join`. Así se conservan también las películas sin reviews válidas; en esos casos, `reviews` queda vacío y luego el pipeline de entrenamiento lo imputa. La limitación es que se resume toda la opinión de usuarios en un único promedio, pero como primera señal textual es simple y estable.

6.1. Logs y diagnóstico del ETL

El ETL incluye varios mensajes de diagnóstico para detectar problemas temprano y entender la calidad de los datos antes de entrenar. Primero valida que existan archivos en los prefijos de S3 para películas y reviews; si faltan datos o credenciales, muestra un mensaje específico para saber si el problema viene de AWS, del bucket o de una corrida pendiente del scraper.

Durante la lectura imprime la cantidad de filas crudas de películas y reviews. Después muestra ejemplos útiles para análisis exploratorio: las 10 películas con mayor rating, las 10 con menor rating y las primeras reviews crudas. Estos logs permiten revisar rápidamente si los datos parecen razonables, si hay ratings fuera de escala o si las reviews llegan con los campos esperados.

Luego de deduplicar, el ETL informa cuántas películas quedan y muestra un resumen de nulos por dataset. Esto ayuda a identificar columnas incompletas, problemas del scraper o señales que luego habrá que imputar. Al final, informa cuántas filas sin `imdb_rating` se eliminaron, cuántas películas quedan en el dataset final y si todavía existen columnas con nulos. Con esa información se pueden sacar conclusiones simples antes de entrenar, por ejemplo si el dataset tiene pocas reviews útiles, si hay muchos registros sin target o si la extracción necesita repetirse.

Campo del JSON procesado	Descripción
<code>imdb_id</code>	Identificador de IMDb; se conserva para trazabilidad, pero no entra al modelo.
<code>title</code>	Título de la película.
<code>year</code>	Año de estreno.
<code>votes</code>	Cantidad de votos.
<code>directors</code>	Lista normalizada de directores.
<code>main_cast</code>	Lista normalizada de actores principales.
<code>reviews</code>	Sentimiento promedio de reviews calculado con VADER y normalizado a escala 0–1.
<code>imdb_rating</code>	Variable objetivo, normalizada a escala 0–1.

7. Feature engineering

El feature engineering del modelo ocurre principalmente en `training/train.py`, después de cargar `training_dataset.json`. El ETL deja los datos limpios y agregados; el entrenamiento convierte esos campos en representaciones numéricas aptas para scikit-learn.

Uso de columnas en el entrenamiento actual:

Columna	Uso actual o motivo
<code>imdb_id</code>	Se excluye del entrenamiento porque es un identificador, no una señal predictiva.
<code>imdb_rating</code>	Se excluye de las features porque es el target.
<code>votes</code>	No se excluye en el modelo actual: se transforma a <code>log_votes</code> . Puede ser una fuente de leakage si se quiere simular una predicción previa al estreno o muy temprana, pero es válida para el escenario post-estreno.

<code>title</code>	No se excluye: se vectoriza con TF-IDF usando unigramas y bigramas.
<code>directors</code>	No se excluye: se convierte a <code>directors_text</code> y se codifica con <code>CountVectorizer</code> .
<code>main_cast</code>	No se excluye: se convierte a <code>cast_text</code> y se codifica con <code>CountVectorizer</code> .
<code>plot,</code> <code>genres,</code> <code>runtime,</code> <code>certificate,</code> <code>metascore</code>	El scraper puede recolectar estas columnas, pero no forman parte del <code>training_dataset.json</code> usado por el modelo actual. Quedan como trabajo futuro para mejorar el feature engineering.

7.1. Feature engineering del entrenamiento actual

El documento inicial del proyecto describe una versión de entrenamiento local basada en `training_dataset.json`. En esa versión, `train.py` aplica tres transformaciones principales antes de entrenar. Además, filtra registros sin director, convierte `reviews`, `year` y `votes` a valores numéricos, e imputa y escala las features numéricas dentro del pipeline.

La primera transformación es `log1p(votes)`. La cantidad de votos que recibe una película en IMDb tiene una distribución muy sesgada: unas pocas películas populares acumulan millones de votos, mientras que la mayoría tiene muchos menos. Esa asimetría perjudica a modelos lineales como ElasticNet. Aplicar `log1p` comprime la escala y hace que la variable sea más manejable sin perder poder predictivo.

La segunda transformación convierte las listas de directores y actores en representaciones vectoriales binarias. En el dataset original, `directors` y `main_cast` son listas de strings. Para usarlas en scikit-learn, se concatenan con el separador `|` y luego un `CountVectorizer` binario con un tokenizer por pipes genera una columna por director o actor que aparece al menos en dos películas. El resultado es un encoding multi-hot.

La tercera transformación vectoriza el título de la película con TF-IDF usando unigramas y

bigramas. El título se trata como texto libre y se calculan pesos TF-IDF sobre los 1000 términos más frecuentes, considerando palabras individuales y pares de palabras consecutivas. Esto permite capturar patrones implícitos en el nombre de la película, como referencias a sagas, géneros o épocas.

Vale aclarar que esa versión del dataset no incluye algunas columnas que el scraper sí recolecta, como géneros, metaspcore, certificado de contenido o argumento. Esas features quedan fuera del modelo actual y representan una oportunidad clara de mejora para iteraciones futuras.

8. Entrenamiento del modelo

El entrenamiento actual toma como entrada el archivo local:

```
training/training_dataset.json
```

Como mejora MLOps, el entrenamiento debería leer ese dataset procesado desde S3 o registrar explícitamente qué versión local se utilizó, para mantener trazabilidad entre datos, corrida de entrenamiento y modelo publicado.

El dataset se separa en variables predictoras y target luego de construir las features derivadas:

```
features = [  
    "title",  
    "year",  
    "log_votes",  
    "reviews",  
    "directors_text",  
    "cast_text",  
]  
X = df[features]  
y = df["imdb_rating"]
```

Como primera aproximación se pueden usar modelos de regresión como Linear Regression, Random Forest Regressor, Gradient Boosting Regressor, XGBoost o LightGBM. En la implemen-

tación documentada inicialmente, el modelo entrenado es un ElasticNet dentro de un Pipeline de scikit-learn.

El script `train.py` lee la data procesada, aplica feature engineering y busca los mejores hiperparámetros `alpha` y `l1_ratio` mediante `GridSearchCV` con validación cruzada de 5 folds, optimizando MAE. Luego evalúa sobre un test set del 20 % y reporta MAE, RMSE y R^2 . Además, imprime las mejores y peores predicciones individuales para facilitar el análisis de errores.

8.1. Publicación en W&B

La parte más importante del cierre del entrenamiento es la publicación en Weights & Biases [5]. El script consulta el MAE del modelo actualmente en producción antes de decidir si promover el nuevo modelo. Solo asigna el alias `production` si el nuevo modelo mejora al existente; en caso contrario, lo sube únicamente con `latest`. Esto evita que un reentrenamiento sobre los mismos datos degrade el modelo productivo.

El equipo cuenta con un team en W&B, lo que permite centralizar los modelos fuera de cuentas personales y facilita que todos los integrantes puedan subir, consultar y promover artefactos.

Configuración del registry:

```
Entity: mlprod-obli
Project: imdb-rating
Artifact: imdb-rating-model
Alias: production
```

La idea es usar alias para promover modelos:

Alias	Uso
<code>candidate</code>	Modelo recién entrenado.
<code>staging</code>	Modelo validado para pruebas.
<code>production</code>	Modelo usado por la API.

9. Prevención de data leakage y training-serving skew

En un sistema de Machine Learning en Producción no alcanza con que el modelo tenga buenas métricas offline. También es necesario asegurar que el modelo no aprenda información que no estaría disponible al momento de predecir y que el procesamiento usado en entrenamiento sea el mismo que se usa en inferencia.

9.1. Data leakage

Data leakage ocurre cuando el modelo usa, directa o indirectamente, información del futuro o información demasiado cercana al target. Esto genera métricas artificialmente buenas durante entrenamiento, pero mal desempeño en producción.

Riesgo	Prevención
Usar <code>imdb_rating</code> como feature	Separar explícitamente <code>X</code> e <code>y</code> , dejando <code>imdb_rating</code> solo como target.
Usar <code>votes</code> como predictor	En el modelo actual se usa transformado como <code>log_votes</code> . Si la predicción representa una película nueva o temprana, debería excluirse o reemplazarse por una señal disponible en ese momento.
Usar reviews posteriores al momento de predicción	Definir fecha de corte o escenario de predicción.
Mezclar duplicados entre train y test	Deduplicar por <code>imdb_id</code> antes del split.
Hacer transformaciones mirando todo el dataset	Ajustar imputadores, escaladores y encoders solo con train.
Elegir modelo mirando el test final	Usar train/validation para selección y reservar test para evaluación final.

Para reviews, una decisión importante es distinguir entre predicción pre-estreno o temprana, donde no deberían usarse reviews de usuarios, y predicción post-estreno, donde pueden usarse siempre

que se registre la fecha de corte.

9.2. Training-serving skew

Training-serving skew ocurre cuando los datos que recibe el modelo en producción no pasan por el mismo procesamiento que los datos usados para entrenar. Para reducir este riesgo, la aplicación en producción no reconstruye features manualmente. En su lugar, la UI muestra una lista de películas disponibles y, cuando el usuario selecciona una, la API obtiene los datos ya procesados desde `inference_dataset.json`.

Ese JSON de inferencia debería generarse con el mismo ETL que produce `training_dataset.json`. Por lo tanto, las columnas derivadas, codificaciones, agregaciones de reviews, tratamiento de nulos y tipos de datos nacen del mismo proceso usado para entrenar.

10. Trazabilidad de ML

La trazabilidad permite reconstruir qué datos, código, parámetros y modelo participaron en una predicción o en una versión publicada. En este proyecto se considera versionar experimentos, modelos y datos.

Elemento	Cómo se versiona	Para qué sirve
Experimentos	Corridas en W&B con parámetros, métricas y artefactos asociados	Comparar modelos, justificar la elección final y auditar resultados.
Modelos	Artifacts de W&B con versiones y aliases	Saber qué modelo está desplegado y volver a una versión anterior.
Datos crudos	Parquet en S3 particionados por fecha y corrida	Identificar de qué extracción provienen los datos.
Dataset procesado	<code>training_dataset.json</code> generado por el ETL	Relacionar cada entrenamiento con el dataset exacto utilizado.

Datos de inferencia	<code>inference_dataset.json</code> generado por el ETL	Asegurar consistencia entre entrenamiento e inferencia.
---------------------	---	---

Para que la trazabilidad sea completa, cada corrida de entrenamiento debería guardar el dataset usado, fecha y versión del ETL, parámetros del modelo, métricas, commit de Git, artifact del modelo y lista de features esperadas.

11. Serving del modelo

La aplicación de serving está implementada con FastAPI [1]. Actualmente expone endpoints para:

Endpoint	Descripción
/	Interfaz web.
/status	Health check de la aplicación.
/movies	Lectura de películas desde S3.
/model	Consulta del artifact productivo en W&B.
POST /predict	Recibe una película seleccionada o su <code>imdb_id</code> , busca features en el JSON de inferencia y devuelve el rating estimado.

El endpoint de predicción carga el modelo productivo desde W&B, recupera del JSON de inferencia las features ya procesadas, excluye columnas que no deben usarse como entrada del modelo y devuelve una respuesta consistente.

11.1. Predicciones en landing page

Para hostear la landing page se optó por usar AWS ECS sobre AWS Academy [4]. Detrás de un Application Load Balancer se ejecuta una task construida desde una imagen publicada en AWS ECR, la cual se pusha localmente usando AWS CLI.

Antes de servir la imagen a ECR, se construye localmente a partir del **Dockerfile**. La imagen contiene el contenido estático de la página web y el web service que oficia de backend, utilizando JavaScript en el frontend y Python FastAPI como web engine.

Una vez levantada la task en ECS, la URL del Load Balancer expone el servicio en Internet. El servicio se conecta a W&B para descargar el modelo y hacer inferencia. Además, obtiene películas que ofician de input para la inferencia desde TMDb, sitio similar a IMDb que expone una API gratuita. Las películas se cargan a demanda desde una barra de búsqueda y, al seleccionar una película, se compara el score predicho contra el score real.

11.2. Predicciones batch

Además de predicciones individuales por medio de la landing page, se deja planteado exponer predicciones en modo batch. Esta funcionalidad permitiría recibir un conjunto de películas, ejecutar inferencia sobre todas ellas y devolver un archivo o respuesta agregada con los ratings estimados.

12. Containerización

La aplicación se empaqueta con Docker usando una imagen base de Python. El contenedor instala las dependencias, copia la aplicación FastAPI y expone el puerto 8000. Esto permite ejecutar el mismo artefacto en local y en producción, reduciendo diferencias entre ambientes.

```
docker compose up --build
```

13. Infraestructura como código

La infraestructura cloud se define con Terraform. Esto permite versionar los recursos, recrearlos y documentar la arquitectura de despliegue.

Recursos definidos:

- AWS ECS Cluster.

- Task Definition para Fargate.
- ECS Service.
- Application Load Balancer.
- Target Group con health check en `/status`.
- Security Groups.
- CloudWatch Log Group.
- Política de acceso a S3 para la task.

El despliegue usa una imagen Docker publicada en ECR y luego referenciada desde Terraform.

14. Gestión de secretos

La API necesita credenciales para consultar W&B. Para evitar hardcodear secretos, se usa AWS SSM Parameter Store.

```
wandb-org  
wandb-api-key
```

Si no se encuentran en SSM, la aplicación permite usar variables de entorno como fallback:

```
WANDB_USER  
WANDB_API_KEY
```

Esto aplica una buena práctica de seguridad: separar secretos del código fuente.

15. Observabilidad y monitoreo

La solución incluye un primer nivel de observabilidad:

- Endpoint `/status` como health check.
- Logs de contenedor enviados a CloudWatch.
- Consulta de versión del modelo productivo mediante `/model`.

Mejoras posibles:

- Registrar cantidad de predicciones realizadas.
- Medir latencia del endpoint `/predict`.
- Guardar errores de inferencia.
- Registrar distribución de features de entrada.
- Detectar drift comparando datos nuevos contra datos de entrenamiento.

16. Automatización

El proyecto incluye un `Makefile` para automatizar tareas frecuentes.

```
make scraper-install
make scraper-login
make scraper-run
make etl-run
make scraper-etl
make pipeline
```

Esto permite ejecutar el flujo de ingesta y procesamiento de forma más ordenada y repetible.

17. Conceptos de clase aplicados

Concepto	Aplicación en el proyecto
----------	---------------------------

Pipeline de datos	Scraper + S3 + ETL.
Data lake	S3 como almacenamiento central.
Versionado de datos	Archivos particionados por fecha y corrida.
Feature engineering	Generación de features numéricas, categóricas y textuales.
Prevención de data leakage	Exclusión de variables no disponibles o demasiado cercanas al target.
Training-serving skew	Pipeline y schema compartidos entre entrenamiento e inferencia.
Reproducibilidad	Docker, Terraform y Makefile.
Tracking de experimentos	W&B para métricas y parámetros.
Model registry	Artifacts y aliases en W&B.
Trazabilidad de ML	Versionado de experimentos, modelos, datos y features usadas.
Serving	FastAPI como API de inferencia.
CI/CD potencial	Build de imagen, push a ECR y deploy con Terraform.
Gestión de secretos	AWS SSM Parameter Store.
Observabilidad	Health check y CloudWatch logs.
Despliegue cloud	ECS Fargate + ALB.

18. Trabajo pendiente

Para completar la solución de punta a punta, quedan pendientes los siguientes puntos:

1. Documentar métricas finales del modelo productivo.
2. Incluir capturas del despliegue funcionando.
3. Completar la funcionalidad de predicciones batch.
4. Agregar tests básicos para ETL y API.
5. Registrar drift y métricas operativas del endpoint de predicción.

19. Conclusiones

El proyecto construye una base sólida para una solución de Machine Learning en Producción. No solo contempla el modelo, sino también los elementos necesarios para operarlo: datos versionados, procesamiento automatizado, empaquetado, infraestructura reproducible, despliegue cloud y registro de modelos.

La principal mejora para cerrar el ciclo MLOps completo es consolidar el entrenamiento productivo con el dataset procesado en S3, publicar métricas finales y terminar el flujo de predicciones batch. Aun así, la arquitectura actual ya refleja los componentes centrales de una solución MLOps: ingestión, procesamiento, entrenamiento, registry, serving, despliegue y observabilidad.

Referencias

- [1] FastAPI. Fastapi documentation. <https://fastapi.tiangolo.com/>, 2026.
- [2] Chip Huyen. *Designing Machine Learning Systems*. O'Reilly Media, 2022.
- [3] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 2015.
- [4] Amazon Web Services. Amazon elastic container service documentation. <https://docs.aws.amazon.com/ecs/>, 2026.
- [5] Weights and Biases. Weights and biases documentation. <https://docs.wandb.ai/>, 2026.